DATA STRUCTURES LAB
PYTHON PROGRAMS

Experiment 1: List Operations (Insertion, Deletion, Display)

Program

```
# Experiment 1: List Operations
lst = list(map(int, input("Enter list elements: ").split()))
print("Original list:", lst)

while True:
    print("\n1. Insert")
    print("2. Delete")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter choice: "))

    if choice == 1:
        value = int(input("Enter element: "))
        pos = int(input("Enter position (0-based): "))
        if 0 <= pos <= len(lst):
            lst.insert(pos, value)
            print("Element inserted.")
        else:
            print("Invalid position.")
    elif choice == 2:
        pos = int(input("Enter position (0-based): "))
        if 0 <= pos < len(lst):
            print("Deleted:", lst.pop(pos))
        else:
            print("Invalid position.")
    elif choice == 3:
        print("List:", lst)
    elif choice == 4:
        print("Exiting program.")
        break
```

```
    else:
        print("Invalid choice.")
```

Output

```
Enter list elements: 10 20 30
Original list: [10, 20, 30]
Enter choice: 1
Enter element: 15
Enter position (0-based): 1
Element inserted.
Enter choice: 3
List: [10, 15, 20, 30]
Enter choice: 2
Enter position (0-based): 0
Deleted: 10
Enter choice: 3
List: [15, 20, 30]
Enter choice: 4
Exiting program.
```

Experiment 2: Quick Sort, Merge Sort and Time Complexity

Program

```
# Experiment 2: Quick Sort and Merge Sort
def quick_sort(a):
    if len(a) <= 1:
        return a
    pivot = a[-1]
    left = [x for x in a[:-1] if x <= pivot]
    right = [x for x in a[:-1] if x > pivot]
    return quick_sort(left) + [pivot] + quick_sort(right)

def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    return result + left[i:] + right[j:]

arr = list(map(int, input("Enter list elements: ").split()))

while True:
    print("\n1. Quick Sort")
    print("2. Merge Sort")
    print("3. Compare Time Complexity")
    print("4. Exit")
    choice = int(input("Enter choice: "))
```

```
    if choice == 1:
        print("Sorted list:", quick_sort(arr))
    elif choice == 2:
        print("Sorted list:", merge_sort(arr))
    elif choice == 3:
        print("Quick Sort: Best/Average O(n log n), Worst O(n^2)")
        print("Merge Sort: Best/Average/Worst O(n log n)")
    elif choice == 4:
        break
    else:
        print("Invalid choice.")
```

Output

```
Enter list elements: 8 3 6 1 5
Enter choice: 1
Sorted list: [1, 3, 5, 6, 8]
Enter choice: 2
Sorted list: [1, 3, 5, 6, 8]
Enter choice: 3
Quick Sort: Best/Average O(n log n), Worst O(n^2)
Merge Sort: Best/Average/Worst O(n log n)
Enter choice: 4
```

Experiment 3: Stack, Queue, Circular Queue Using Arrays and Browser History

Program

```
# Experiment 3: Stack, Queue, Circular Queue and Browser History
from collections import deque

stack = []
queue = []
SIZE = 5
cq = [None] * SIZE
front = rear = -1
history = []
current_page = None

while True:
    print("\n1. Stack  2. Queue  3. Circular Queue")
    print("4. Browser History  5. Exit")
    choice = int(input("Enter choice: "))

    if choice == 5:
        break

    if choice == 1:
        print("1. Push  2. Pop  3. Display")
        op = int(input("Operation: "))
        if op == 1:
            stack.append(input("Value: "))
        elif op == 2:
            print("Popped:", stack.pop() if stack else "Stack empty")
        elif op == 3:
            print("Stack:", stack)

    elif choice == 2:
        print("1. Enqueue  2. Dequeue  3. Display")
        op = int(input("Operation: "))
        if op == 1:
            queue.append(input("Value: "))
        elif op == 2:
```

```
        print("Removed:", queue.pop(0) if queue else "Queue empty")
    elif op == 3:
        print("Queue:", queue)

elif choice == 3:
    print("1. Enqueue  2. Dequeue  3. Display")
    op = int(input("Operation: "))
    if op == 1:
        if (rear + 1) % SIZE == front:
            print("Circular queue is full")
        else:
            value = input("Value: ")
            if front == -1:
                front = rear = 0
            else:
                rear = (rear + 1) % SIZE
            cq[rear] = value
    elif op == 2:
        if front == -1:
            print("Circular queue is empty")
        else:
            print("Removed:", cq[front])
            cq[front] = None
            if front == rear:
                front = rear = -1
            else:
                front = (front + 1) % SIZE
    elif op == 3:
        if front == -1:
            print("Circular queue is empty")
        else:
            i = front
            while True:
                print(cq[i], end=" ")
                if i == rear:
                    break
                i = (i + 1) % SIZE
```

```
            print()

    elif choice == 4:
        print("1. Visit  2. Back  3. Display")
        op = int(input("Operation: "))
        if op == 1:
            if current_page is not None:
                history.append(current_page)
            current_page = input("URL: ")
            print("Current page:", current_page)
        elif op == 2:
            if history:
                current_page = history.pop()
                print("Current page:", current_page)
            else:
                print("No previous page")
        elif op == 3:
            print("Previous pages:", history)
            print("Current page:", current_page)
    else:
        print("Invalid choice")
```

Output

```
Enter choice: 1
Operation: 1
Value: 10
Enter choice: 1
Operation: 3
Stack: ['10']
Enter choice: 2
Operation: 1
Value: A
Enter choice: 2
Operation: 3
Queue: ['A']
Enter choice: 4
Operation: 1
URL: example.com

Current page: example.com
Enter choice: 5
```

Experiment 4: Evaluate Postfix Notation

Program

```
# Experiment 4: Postfix Evaluation
def evaluate_postfix(expression):
    stack = []
    for token in expression.split():
        if token.lstrip("-").isdigit():
            stack.append(int(token))
        else:
            if len(stack) < 2:
                raise ValueError("Invalid postfix expression")
            b = stack.pop()
            a = stack.pop()
            if token == "+":
                stack.append(a + b)
            elif token == "-":
                stack.append(a - b)
            elif token == "*":
                stack.append(a * b)
            elif token == "/":
                stack.append(a / b)
            elif token == "^":
                stack.append(a ** b)
            else:
                raise ValueError("Invalid operator")
    if len(stack) != 1:
        raise ValueError("Invalid postfix expression")
    return stack[0]

while True:
    print("\n1. Evaluate Postfix")
    print("2. Exit")
    choice = int(input("Enter choice: "))
    if choice == 1:
        expression = input("Enter postfix expression (space-separated): ")
        try:
            print("Result:", evaluate_postfix(expression))
```

```
        except (ValueError, ZeroDivisionError) as e:
            print("Error:", e)
    elif choice == 2:
        break
    else:
        print("Invalid choice")
```

Output

```
1. Evaluate Postfix
2. Exit
Enter choice: 1
Enter postfix expression (space-separated): 5 3 + 2 *
Result: 16
1. Evaluate Postfix
2. Exit
Enter choice: 2
```

Experiment 5: Convert Infix Notation to Postfix Notation

Program

```
# Experiment 5: Infix to Postfix
def infix_to_postfix(expression):
    stack = []
    output = []
    precedence = {"+": 1, "-": 1, "*": 2, "/": 2, "^": 3}

    for token in expression.split():
        if token.isalnum():
            output.append(token)
        elif token == "(":
            stack.append(token)
        elif token == ")":
            while stack and stack[-1] != "(":
                output.append(stack.pop())
            if not stack:
                raise ValueError("Mismatched parentheses")
            stack.pop()
        elif token in precedence:
            while (stack and stack[-1] != "(" and
                   (precedence[stack[-1]] > precedence[token] or
                    (precedence[stack[-1]] == precedence[token]
                     and token != "^"))):
                output.append(stack.pop())
            stack.append(token)
        else:
            raise ValueError("Invalid token")

    while stack:
        if stack[-1] == "(":
            raise ValueError("Mismatched parentheses")
        output.append(stack.pop())
    return " ".join(output)

while True:
    print("\n1. Convert Infix to Postfix")
```

```
print("2. Exit")
choice = int(input("Enter choice: "))
if choice == 1:
    expression = input("Enter infix expression with spaces: ")
    try:
        print("Postfix:", infix_to_postfix(expression))
    except ValueError as e:
        print("Error:", e)
elif choice == 2:
    break
else:
    print("Invalid choice")
```

Output

```
Enter choice: 1
Enter infix expression with spaces: A + B * C
Postfix: A B C * +
Enter choice: 2
```

Experiment 6: Linked List Operations (Insertion, Deletion, Display)

Program

```
# Experiment 6: Singly Linked List
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        node = Node(data)
        if self.head is None:
            self.head = node
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = node
        print("Element inserted")

    def delete(self, value):
        temp = self.head
        if temp and temp.data == value:
            self.head = temp.next
            print("Element deleted")
            return
        while temp and temp.next:
            if temp.next.data == value:
                temp.next = temp.next.next
                print("Element deleted")
                return
            temp = temp.next
        print("Element not found")
```

```
    def display(self):
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")

ll = LinkedList()
while True:
    print("\n1. Insert  2. Delete  3. Display  4. Exit")
    choice = int(input("Enter choice: "))
    if choice == 1:
        ll.insert(int(input("Enter value: ")))
    elif choice == 2:
        ll.delete(int(input("Enter value: ")))
    elif choice == 3:
        ll.display()
    elif choice == 4:
        break
    else:
        print("Invalid choice")
```

Output

```
Enter choice: 1
Enter value: 10
Element inserted
Enter choice: 1
Enter value: 20
Element inserted
Enter choice: 3
10 -> 20 -> None
Enter choice: 2
Enter value: 10
Element deleted
Enter choice: 3
20 -> None
Enter choice: 4
```

Experiment 7: Circular Linked List Operations

Program

```
# Experiment 7: Circular Linked List
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularLinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        node = Node(data)
        if self.head is None:
            self.head = node
            node.next = node
        else:
            temp = self.head
            while temp.next != self.head:
                temp = temp.next
            temp.next = node
            node.next = self.head
        print("Element inserted")

    def delete(self, value):
        if self.head is None:
            print("List is empty")
            return
        current = self.head
        previous = None
        while True:
            if current.data == value:
                if previous is None:
                    if current.next == self.head:
                        self.head = None
                    else:
```

```
                last = self.head
                while last.next != self.head:
                    last = last.next
                self.head = current.next
                last.next = self.head
            else:
                previous.next = current.next
            print("Element deleted")
            return
        previous = current
        current = current.next
        if current == self.head:
            break
    print("Element not found")

  def display(self):
    if self.head is None:
        print("List is empty")
        return
    temp = self.head
    while True:
        print(temp.data, end=" -> ")
        temp = temp.next
        if temp == self.head:
            break
    print("(back to head)")

cll = CircularLinkedList()
while True:
  print("\n1. Insert  2. Delete  3. Display  4. Exit")
  choice = int(input("Enter choice: "))
  if choice == 1:
    cll.insert(int(input("Enter value: ")))
  elif choice == 2:
    cll.delete(int(input("Enter value: ")))
  elif choice == 3:
    cll.display()
```

```
    elif choice == 4:
        break
    else:
        print("Invalid choice")
```

Output

```
Enter choice: 1
Enter value: 10
Element inserted
Enter choice: 1
Enter value: 20
Element inserted
Enter choice: 3
10 -> 20 -> (back to head)
Enter choice: 4
```

Experiment 8: Stack, Queue and Circular Queue Using Linked Lists

Program

```
# Experiment 8: Stack and Queues Using Linked Lists
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedStructure:
    def __init__(self, circular=False):
        self.front = None
        self.rear = None
        self.circular = circular

    def insert(self, data):
        node = Node(data)
        if self.front is None:
            self.front = self.rear = node
            if self.circular:
                node.next = node
        else:
            self.rear.next = node
            self.rear = node
            if self.circular:
                self.rear.next = self.front
        print("Element inserted")

    def delete(self):
        if self.front is None:
            print("Structure is empty")
            return
        print("Deleted:", self.front.data)
        if self.front == self.rear:
            self.front = self.rear = None
        else:
            self.front = self.front.next
            if self.circular:
```

```
            self.rear.next = self.front

    def display(self):
        if self.front is None:
            print("Structure is empty")
            return
        temp = self.front
        while True:
            print(temp.data, end=" -> ")
            if temp == self.rear:
                break
            temp = temp.next
        print("(back to front)" if self.circular else "None")

class Stack(LinkedStructure):
    def insert(self, data):
        node = Node(data)
        node.next = self.front
        self.front = node
        if self.rear is None:
            self.rear = node
        print("Element pushed")

    def delete(self):
        if self.front is None:
            print("Stack is empty")
            return
        print("Popped:", self.front.data)
        self.front = self.front.next
        if self.front is None:
            self.rear = None

stack = Stack()
queue = LinkedStructure()
circular_queue = LinkedStructure(True)

while True:
```

```
    print("\n1. Stack  2. Queue  3. Circular Queue  4. Exit")
    choice = int(input("Enter choice: "))
    if choice == 4:
        break
    structures = {1: stack, 2: queue, 3: circular_queue}
    if choice not in structures:
        print("Invalid choice")
        continue
    structure = structures[choice]
    print("1. Insert/Push  2. Delete/Pop  3. Display")
    op = int(input("Enter operation: "))
    if op == 1:
        structure.insert(int(input("Enter value: ")))
    elif op == 2:
        structure.delete()
    elif op == 3:
        structure.display()
    else:
        print("Invalid operation")
```

Output

```
Enter choice: 1
Enter operation: 1
Enter value: 10
Element pushed
Enter choice: 1
Enter operation: 3
10 -> None
Enter choice: 2
Enter operation: 1
Enter value: 20
Element inserted
Enter choice: 2
Enter operation: 3
20 -> None
Enter choice: 4
```

Experiment 9: Concatenation of Linked Lists and Polynomial Addition

Program

```
# Experiment 9: Linked List Concatenation and Polynomial Addition
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

def create_list():
    head = tail = None
    n = int(input("Enter number of elements: "))
    for _ in range(n):
        node = Node(int(input("Enter value: ")))
        if head is None:
            head = tail = node
        else:
            tail.next = node
            tail = node
    return head

def display_list(head):
    while head:
        print(head.data, end=" -> ")
        head = head.next
    print("None")

def concatenate(first, second):
    if first is None:
        return second
    temp = first
    while temp.next:
        temp = temp.next
    temp.next = second
    return first

def create_polynomial():
    poly = {}
```

```
    n = int(input("Enter number of terms: "))
    for _ in range(n):
        coef = int(input("Enter coefficient: "))
        exp = int(input("Enter exponent: "))
        poly[exp] = poly.get(exp, 0) + coef
    return poly

def display_polynomial(poly):
    terms = [f"{poly[e]}x^{e}" for e in sorted(poly, reverse=True) if poly[e] != 0]
    print(" + ".join(terms) if terms else "0")

while True:
    print("\n1. Concatenate Linked Lists")
    print("2. Add Polynomials")
    print("3. Exit")
    choice = int(input("Enter choice: "))
    if choice == 1:
        print("First list:")
        first = create_list()
        print("Second list:")
        second = create_list()
        print("Concatenated list:")
        display_list(concatenate(first, second))
    elif choice == 2:
        print("First polynomial:")
        p1 = create_polynomial()
        print("Second polynomial:")
        p2 = create_polynomial()
        result = p1.copy()
        for exp, coef in p2.items():
            result[exp] = result.get(exp, 0) + coef
        print("Sum of polynomials:")
        display_polynomial(result)
    elif choice == 3:
        break
    else:
        print("Invalid choice")
```

Output

```
Enter choice: 1
First list:
Enter number of elements: 2
Enter value: 10
Enter value: 20
Second list:
Enter number of elements: 2
Enter value: 30
Enter value: 40
Concatenated list:
10 -> 20 -> 30 -> 40 -> None

Enter choice: 2
First polynomial:
Enter number of terms: 2
Enter coefficient: 3
Enter exponent: 2
Enter coefficient: 2
Enter exponent: 1
Second polynomial:
Enter number of terms: 2
Enter coefficient: 4
Enter exponent: 2
Enter coefficient: 5
Enter exponent: 0
Sum of polynomials:
7x^2 + 2x^1 + 5x^0
Enter choice: 3
```

Experiment 10: Doubly Linked List Operations

Program

```
# Experiment 10: Doubly Linked List
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        node = Node(data)
        if self.head is None:
            self.head = node
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = node
            node.prev = temp
        print("Element inserted")

    def delete(self, value):
        temp = self.head
        while temp and temp.data != value:
            temp = temp.next
        if temp is None:
            print("Element not found")
            return
        if temp.prev:
            temp.prev.next = temp.next
        else:
            self.head = temp.next
        if temp.next:
```

```
            temp.next.prev = temp.prev
        print("Element deleted")

    def display(self):
        temp = self.head
        last = None
        print("Forward:", end=" ")
        while temp:
            print(temp.data, end=" <-> ")
            last = temp
            temp = temp.next
        print("None")
        print("Backward:", end=" ")
        while last:
            print(last.data, end=" <-> ")
            last = last.prev
        print("None")

dll = DoublyLinkedList()
while True:
    print("\n1. Insert  2. Delete  3. Display  4. Exit")
    choice = int(input("Enter choice: "))
    if choice == 1:
        dll.insert(int(input("Enter value: ")))
    elif choice == 2:
        dll.delete(int(input("Enter value: ")))
    elif choice == 3:
        dll.display()
    elif choice == 4:
        break
    else:
        print("Invalid choice")
```

Output

```
Enter choice: 1
Enter value: 10
Element inserted
Enter choice: 1
```

Enter value: 20
Element inserted
Enter choice: 3
Forward: 10 <-> 20 <-> None
Backward: 20 <-> 10 <-> None
Enter choice: 4